DATA AND SECURITY IMPLEMENTATION REPORT

UDP-Based Secure File Transfer System

Project	UDP-Based File Transfer System
Role	Data & Security Specialist
Programming Language	Python
Communication Protocol	UDP
Security Algorithm	AES-256-GCM
Integrity Verification	SHA-256

Prepared as a technical summary of the Data and Security component, covering UDP file transfer, reliability mechanisms, encryption, integrity verification, and security testing.

1. Introduction

The purpose of this component is to implement a reliable and secure file transfer mechanism using the User Datagram Protocol (UDP).

Unlike TCP, UDP does not guarantee packet delivery, ordering, or retransmission. Therefore, additional mechanisms are required to ensure that files can be transferred correctly and securely.

The Data & Security component focuses on the following objectives:

- Implement file chunking to divide large files into smaller data packets.
- Implement acknowledgment (ACK) and retransmission mechanisms to improve transfer reliability.
- Protect file confidentiality using AES-256-GCM encryption.
- Verify file integrity using SHA-256 hashing.
- Test the system under normal conditions and incorrect encryption-key conditions.

2. System Architecture

The system follows a Client-Server architecture. The Client reads the original file, encrypts the data, divides the encrypted data into chunks, and transmits those chunks to the Server through UDP. The Server receives the encrypted chunks, reconstructs the encrypted data, decrypts it, verifies its integrity, and saves the recovered original file.

2.1 Main Components

Client

- Reads the original file.
- Calculates the SHA-256 hash of the original data.
- Encrypts the file using AES-256-GCM.
- Splits encrypted data into 1024-byte chunks.
- Sends chunks to the Server.
- Waits for ACK responses and retransmits packets when necessary.

Server

- Listens for incoming UDP packets.
- Receives START, CHUNK, and END messages.
- Stores received chunks according to their sequence numbers.
- Sends ACK messages to the Client.
- Reassembles the encrypted data.
- Decrypts the data using AES-256-GCM.
- Verifies the SHA-256 hash and saves the recovered original file.

3. UDP File Transfer Implementation

3.1 Communication Protocol

The application defines three main message types:

- **START:** The Client sends the filename, total number of chunks, and expected SHA-256 hash to the Server.

- **CHUNK:** The Client sends an individual data chunk with its sequence number.
- **END:** The Client informs the Server that all chunks have been transmitted and requests final file processing.

The Server responds with the following message types:

- **READY:** The Server is ready to receive the file.
- **ACK:** A specific chunk has been received.
- **DONE:** The file has been received, decrypted, and verified successfully.
- **ERROR:** An error occurred during file processing.

3.2 File Chunking

To support file transmission over UDP, the encrypted file data is divided into smaller chunks. The implementation uses a chunk size of 1024 bytes. Each chunk contains a sequence number and a binary payload. The sequence number allows the Server to identify the correct position of each chunk during reconstruction. After receiving all chunks, the Server reconstructs the encrypted data in the correct order.

3.3 Reliability Mechanism: ACK and Retransmission

UDP does not provide built-in reliable delivery. Therefore, the system implements a Stop-and-Wait Automatic Repeat reQuest (ARQ) mechanism.

- The Client sends one data chunk.
- The Server receives the chunk and sends an ACK containing its sequence number.
- The Client checks whether the received ACK matches the transmitted chunk.
- If the ACK is received, the Client sends the next chunk.
- If no ACK arrives before the timeout, the Client retransmits the chunk.
- The Client stops attempting transmission after reaching the maximum retry limit.

The implementation uses a timeout of 2 seconds and a maximum of 5 attempts. The Server also detects duplicate chunks and acknowledges them without storing the same chunk again. This mechanism improves reliability when packets or acknowledgments are lost.

4. Data Security Implementation

4.1 AES-256-GCM Encryption

To protect file confidentiality, the system uses AES-256-GCM (Advanced Encryption Standard with a 256-bit key in Galois/Counter Mode). AES-GCM provides data confidentiality through encryption and data authentication through an authentication tag. It can detect unauthorized modification or an incorrect decryption key.

Before transmission, the Client encrypts the entire original file. The encrypted output consists of a 12-byte random nonce, the encrypted ciphertext, and a 16-byte authentication tag. The nonce is generated using a cryptographically secure random number generator for each encryption operation. The encrypted data is then divided into 1024-byte chunks and transmitted through UDP.

4.2 Key Management

The system uses a pre-shared key (PSK) approach. The Client and Server must possess the same 32-byte AES key to encrypt and decrypt the file successfully. The encryption key is stored locally and is not transmitted through the UDP communication channel.

Separate key files are used during the incorrect-key security test to simulate a key mismatch between the Client and Server. For a real-world deployment, a secure key distribution and key management mechanism would be

required.

4.3 AES-GCM Decryption

After receiving all encrypted chunks, the Server reconstructs the encrypted data. The Server extracts the nonce and uses its AES-256 key to decrypt the ciphertext. AES-GCM automatically verifies the authentication tag during decryption. If the key is incorrect or the encrypted data has been modified, authentication fails and the decryption operation is rejected. The Server does not save the newly received file when decryption fails.

5. Data Integrity Verification

5.1 SHA-256 Hashing

The system uses SHA-256 to verify that the received file is identical to the original file. The Client calculates the SHA-256 hash before encryption. After receiving and decrypting the file, the Server calculates the SHA-256 hash of the recovered original data. The Server compares the calculated hash with the expected hash received in the START message. If both hashes match, the file passes the integrity verification.

5.2 Verification Process

- Read the original file on the Client.
- Calculate its SHA-256 hash.
- Encrypt and transmit the file.
- Reassemble and decrypt the received data on the Server.
- Calculate the SHA-256 hash of the decrypted file.
- Compare both hash values.

A matching hash indicates that the received file content is identical to the original file, assuming no practical SHA-256 collision.

6. Testing and Evaluation

6.1 Functional Test Cases

Test ID	Test Description	Expected Result	Actual Result
TC-01	Establish UDP Client-Server communication	Successful handshake	Passed
TC-02	Divide file into 1024-byte chunks	Correct chunk generation	Passed
TC-03	Transfer file using UDP	All chunks received	Passed
TC-04	Simulate ACK loss	Client retransmits chunk	Passed
TC-05	Detect duplicate chunks	Duplicate not stored again	Passed
TC-06	Encrypt file using AES-256-GCM	Encryption successful	Passed
TC-07	Decrypt with correct key	Decryption successful	Passed
TC-08	Verify file using SHA-256	Both hashes match	Passed
TC-09	Use an incorrect AES key	Authentication failure	Passed
TC-10	Perform final file transfer	Original file recovered	Passed

6.2 Incorrect AES Key Test

Objective: This test evaluates whether the system can reject encrypted data when the Client and Server use different AES keys.

Test Procedure:

- Generate separate 32-byte keys for the Client and Server.
- Encrypt the original file using the Client's key.
- Transfer encrypted chunks to the Server through UDP.
- Attempt to decrypt the received data using the Server's different key.

Expected Result: AES-GCM authentication should fail, and the Server should reject the file.

Actual Result: The Server reported `AES-GCM authentication failed`. The decryption operation was rejected, and the newly received file was not saved.

Conclusion: The test passed. AES-256-GCM successfully detected the incorrect key and prevented unauthorized decryption.

6.3 Final Integrity Test

After restoring the same AES key on both sides, the file transfer was performed again. The SHA-256 hash of the original file was compared with the SHA-256 hash of the received file. Both hash values matched. The final file transfer and integrity verification were successful.

7. Limitations and Future Improvements

- **Key Management:** Implement a secure key exchange mechanism instead of manually sharing the pre-shared key.
- **Transfer Efficiency:** Replace Stop-and-Wait ARQ with a Sliding Window protocol to improve throughput.
- **Scalability:** Improve the Server to support multiple concurrent clients.
- **Transfer Recovery:** Implement persistent transfer state to resume interrupted file transfers.
- **Protocol Security:** Add stronger session management and protection against unauthorized clients and replayed messages.

8. Conclusion

The Data & Security component successfully implements a UDP-based file transfer mechanism with additional reliability, confidentiality, and integrity protection. File chunking and Stop-and-Wait ARQ improve the reliability of UDP communication. AES-256-GCM protects the confidentiality and authenticity of transferred data, while SHA-256 verifies that the recovered file matches the original content.

The testing results demonstrate that the system can successfully transfer and recover files under normal conditions and reject decryption attempts using an incorrect AES key. Therefore, the implementation meets the main functional and security objectives of the Data & Security component.